Lab 2A: Git and GitHub, Step by Step

Nov 8, 2026

Type each command, press Enter, and check your output matches before moving on. Edit files in VS Code; use Git Bash only for the git commands.

Before you start

1. Open **Git Bash** from the Start menu.
2. Check Git is installed:

```
git --version
```

You should see something like `git version 2.40.1.windows.1`. If you see `command not found`, raise your hand.

3. Tell Git who you are. Use the **same email as your GitHub account**. On a shared lab PC, do this every class, because the last student's name may still be saved.

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --global init.defaultBranch main
git config --global --list
```

Check the list shows your name, your email and `init.defaultbranch=main`. Ignore any other lines.

Tips for the terminal

- Type one command at a time and press Enter.
- Every Git command starts with `git`. `command not found` usually means a typo in the first word.
- To paste in Git Bash, use **right-click > Paste** or **Shift + Insert**, not Ctrl + V.
- **Ctrl + C** cancels the current line. It does not copy in Git Bash.
- Yellow `warning:` text is information. Only `error:` or `fatal:` means something failed.

Part 1: Your first repo and commits

The daily loop: edit a file, then `git status`, `git add`, `git commit`. Red in `git status` means changed but not staged; green means staged and ready to commit.

1. Make a project folder and turn it into a repo

```
cd ~/Desktop
mkdir git-practice
cd git-practice
git init
git status
```

Your prompt should now end with `(main)`. If it doesn't, you're in the wrong folder.

2. Create a file. In VS Code, use File > Open Folder and open `git-practice`. Create a file called `README.md`, type `# Git Practice`, and save with Ctrl + S.

```
git status
```

README.md appears in **red** under "Untracked files".

3. Stage it and commit it

```
git add README.md
git status
git commit -m "Add README"
git log --oneline
```

After `git add` the file is **green**. After the commit, `git log` shows one line like `59d0e70 (HEAD -> main) Add README`. Your ID will be different; that's normal.

4. Change the file and commit again. In VS Code, add a second line to README.md and save.

```
git status
git diff
git add README.md
git commit -m "Add description to README"
git log --oneline
```

`git diff` shows your new line with a `+`. `git log` now shows two commits, newest on top.

5. Add a .gitignore. Some files must never go to GitHub, especially `.env` files that hold secret keys. In VS Code, create a file called exactly `.gitignore` containing these three lines, and save:

```
.venv/
.env
__pycache__/
```

```
git add .gitignore
git commit -m "Add gitignore"
```

6. Prove it works. In VS Code, create a file called `.env` containing `SECRET_KEY=abc123` and save it. Then:

```
git status
```

You should see `nothing to commit, working tree clean`. The `.env` file is there, but Git ignores it.

Part 2: Branches, merging and conflicts

A **branch** is a separate line of work. `main` should always work; new work happens on a branch and is merged back when it's ready. Always **save your files in VS Code before switching branches**.

7. Create a branch and commit on it

```
git switch -c feature/greeting
```

Your prompt now shows `(feature/greeting)`. In VS Code, create `hello.py` containing `print("hello")` and save.

```
git add hello.py
git commit -m "Add greeting script"
```

8. Switch back to main

```
git switch main
```

Look in VS Code: `hello.py` has **disappeared**. It isn't deleted; it only exists on the other branch. If VS Code still shows it in a tab, close the tab.

9. Merge the branch into main

```
git merge feature/greeting
git branch -d feature/greeting
git log --oneline
```

`hello.py` is back, now on main. The word `Fast-forward` just means main simply moved forward to include the branch's work.

10. Create a merge conflict on purpose. A conflict happens when two branches change the same line differently.

```
git switch -c feature/a
```

In VS Code, create `note.txt` containing `version A` and save.

```
git add note.txt
git commit -m "Add note version A"
git switch main
```

In VS Code, create `note.txt` again, this time containing `version B`, and save.

```
git add note.txt
git commit -m "Add note version B"
git merge feature/a
```

You'll see `CONFLICT ... Merge conflict in note.txt`. **This is normal.** Git is asking you to choose.

11. Resolve the conflict. Open `note.txt` in VS Code:

```
<<<<<<< HEAD
version B
=======
version A
>>>>>>> feature/a
```

The top part is your current branch (main); the bottom part is the branch being merged in. Click **Accept Both Changes** above the conflict in VS Code (or edit it by hand), make sure the `<<<<<<<`, `=======` and `>>>>>>>` lines are gone, and save.

```
git add note.txt
git commit -m "Resolve merge conflict"
git log --oneline --graph
```

The graph shows the two lines of history joining.

Part 3: Push to GitHub

12. Create an empty repo on GitHub

1. Sign in at github.com and click **New repository** (the + at the top right).
2. Name it `git-practice` and choose **Public**.
3. **Do not tick "Add a README".** Your laptop already has one.
4. Click **Create repository**.

13. Connect and push. Replace `YOUR-USERNAME` with your GitHub username.

```
git remote add origin https://github.com/YOUR-USERNAME/git-practice.git
git branch -M main
git push -u origin main
```

A browser window should open asking you to sign in to GitHub. Approve it. **Your GitHub password will not work if it's asked for in the terminal**; tell the instructor.

Refresh your GitHub page. Your files and commits are now online.

What each line does: `remote add origin` saves the GitHub address under the nickname `origin`. `push -u origin main` uploads main and remembers the link, so next time `git push` is enough.

Push sends up, pull brings down: `git push` uploads your commits; `git pull` downloads new commits from GitHub.

Stretch: a pull request

A pull request (PR) is how teams propose changes: "here's my branch, please review it and merge it".

```
git switch -c update-readme
```

In VS Code, add a line to README.md and save.

```
git add README.md
git commit -m "Update README"
git push -u origin update-readme
```

On GitHub, click the yellow **Compare & pull request** banner, then **Create pull request**. Ask your neighbour to look at it, then click **Merge pull request** and **Confirm merge**. Bring the change back to your PC:

```
git switch main
git pull
```

You're done when your `git-practice` repo is on GitHub with at least 4 commits and a `.gitignore`. Post the link in the class group.

If something goes wrong

You see	What it means	Fix
`command not found`	Typo in the first word	Every command starts with `git`
Strange characters like `^[[200~`	A paste glitch	Clear the line and type the command
A screen full of `~` after `git commit`	You forgot `-m`, so the Vim editor opened	Press Esc, type `:wq`, press Enter
`fatal: not a git repository`	You're in the wrong folder	`cd ~/Desktop/git-practice`
`Author identity unknown`	Git doesn't know who you are	Run the `git config` lines from Before you start
`nothing to commit`	File not saved, or not added	Save in VS Code, then `git status`
`LF will be replaced by CRLF`	Windows line endings	Harmless, ignore it
`error: src refspec main does not match any`	Pushing before any commit	Make a commit first

You see	What it means	Fix
`! [rejected] ... (fetch first)`	GitHub has commits you don't	`git pull --rebase origin main`, then push again
`remote origin already exists`	You ran `remote add` twice	`git remote set-url origin <correct-url>`
`Password authentication is not supported`	GitHub doesn't accept passwords in the terminal	Ask the instructor

Before you leave the lab PC

This PC is shared, so remove your GitHub login before you go:

1. Sign out of github.com in the browser.
2. Open the Start menu, search **Credential Manager**, choose **Windows Credentials**, find the entry starting with `git:https://github.com`, and click **Remove**.

Your work is safe on GitHub. Next time, use `git clone` with your repo's URL to get it back on any PC.